UNIVERSITÉ DE LORRAINE | IUT nancy Charlemagne

BUT informatique 3A, parcours DACS

(pré-)SAÉ de projet tuteuré

Projet audit et déploiement
19 octobre 2026

1 Objectif

Dans ce projet, il s'agit de développer une solution d'audit de systèmes Linux, afin de refaire le point sur les commandes de base, notamment celles permettant d'obtenir des informations sur de tels systèmes. Le projet se déroule en trois étapes progressives :

1. manipulation et extraction d'informations système via les commandes Unix,
2. automatisation et enrichissement par des scripts Ruby,
3. conteneurisation de l'environnement et déploiement *via* Docker.

Le travail s'effectue en binôme (arrangez-vous entre vous pour en déterminer les membres). Ce sujet est également un prétexte pour parfaire vos notions sur les différentes commandes et technologies à utiliser. Il est vivement conseillé de profiter de cette occasion pour « creuser » les concepts utilisés. L'environnement Linux est **obligatoire** pour les travaux demandés. Les distributions cibles sont **Debian** et **Ubuntu** : votre solution doit fonctionner sur ces deux distributions.

L'évaluation finale se composera du rendu des ressources demandées ainsi que d'une mini-soutenance (si possible) pour répondre du travail réalisé et de la culture générale technique sous-jacente. À ce titre, il est important que vous testiez et compreniez les commandes utilisées : se reposer sur un agent IA (interdit par ailleurs par le règlement intérieur) ne vous permettra pas de vous approprier les compétences visées. L'évaluation se concentrera sur ce que vous avez **compris**, pas sur ce que vous avez **produit**.

2 Descriptif des étapes à réaliser

❶ Audit système

Objectif : savoir localiser, collecter et formater des informations système pertinentes à l'aide des commandes Unix usuelles.

Dans cette partie, il s'agit d'être capable de retrouver les informations système suivantes au moment où elles sont récupérées :

- nom de la machine, distribution (nom et version), version du noyau,
- *uptime*, charge moyenne, mémoire et swap disponibles et utilisés,
- liste des interfaces réseau : adresses MAC et IP (v4 et v6) associées,
- liste des utilisateurs humains existants, en distinguant ceux actuellement connectés,
- espace disque par partition (disponible, utilisé),
- les N processus les plus consommateurs de CPU et les N plus consommateurs de mémoire (N étant paramétrable),

— liste des connexions réseau (TCP et UDP, ports en écoute et connexions établies) avec, pour chacune, le processus associé,
— présence et statut de certains services clés (ex. : `ssh`, `cron`, `docker`) : actif ou non (`active`, `inactive`, `failed`), activé ou non au démarrage (`enabled`, `disabled`).

Commentaires sur les travaux demandés :
— les commandes utilisées doivent être standards (`uname`, `free`, `df`, `ps`, `ip`, `ss`, `systemctl`, *etc.*),
— pour chaque information demandée, il faut prévoir un descriptif commenté de l'approche adoptée et de l'usage réalisé des commandes (quels paramètres ? sur quelle ressource ? droits `root` nécessaires ou non ? *etc.*) Inutile d'être trop verbeux sur les informations facilement récupérables. Certaines d'entre elles peuvent être abordées *via* différents points de vue : justifier l'approche que vous avez finalement suivie.

Indications spécifiques à Debian et Ubuntu
— La source standard pour identifier la distribution est le fichier `/etc/os-release`. La commande `lsb_release` dépend du paquet `lsb-release`, absent des installations minimales et de la plupart des images Docker.
— La plage des UID attribués aux utilisateurs « humains » est définie par `UID_MIN` et `UID_MAX` dans `/etc/login.defs` : ne pas coder en dur la valeur 1000. Attention au compte `nobody` (UID 65534).
— Sous Debian et Ubuntu, le service SSH s'appelle `ssh` (`sshd` n'en est qu'un alias) et le service de planification `cron`. Depuis Ubuntu 22.10, SSH est par défaut activé à la demande *via* `ssh.socket` : `ssh.service` peut donc être inactif alors que le serveur répond. Votre audit doit en tenir compte.
— `netstat` (paquet `net-tools`) n'est plus installé par défaut : utiliser `ss` (paquet `iproute2`). Pour voir les processus appartenant à d'autres utilisateurs, les droits `root` sont nécessaires.

2 Automatisation *via* un script Ruby

Objectif : agréger la récupération de toutes les informations précédentes *via* un script, gérer le formatage du résultat.

Dans cette partie, il s'agit de réaliser un script Ruby *paramétrable*, permettant l'automatisation de la récupération de toutes les informations demandées page 1. Ce script devra supporter différents paramètres (aucun dialogue interactif n'est autorisé), permettant :
— de récupérer lors de son exécution toutes les informations évoquées dans la première partie,
— au choix d'afficher les informations récupérées (avec un formatage ergonomique) ou d'enregistrer ces informations dans un fichier au format JSON (incluant la date et l'heure de l'audit),
— de fixer la valeur de N (processus) et la liste des services à vérifier,
— de lire les fichiers système (`/etc/os-release`, `/etc/passwd`, *etc.*) sous un répertoire racine paramétrable (par défaut `/`) : cette option sera indispensable dans la partie de la page 3.

Commentaires sur les travaux demandés :
— le script doit être versionné en utilisant `git` : l'adresse du dépôt correspondant (public au moment du rendu final) devra être communiquée lors du rendu,
— le script doit fonctionner sans les droits `root` : les informations inaccessibles sont alors signalées comme telles, sans que le script ne s'interrompe,
— le code doit être propre et commenté.

Indications pour simplifier la mise en œuvre
— La bibliothèque standard de Ruby suffit : `optparse` (analyse des options), `json` (`JSON.pretty_generate`) et `open3` (`Open3.capture3` pour exécuter une commande et récupérer sa sortie standard, sa sortie d'erreur et son code de retour). Aucune *gem* externe n'est nécessaire, ce qui simplifiera également l'image Docker.
— Préférez les formats de sortie faciles à analyser : `ip -j` produit directement du JSON, `ps -eo … --sort=…`, `df -P` ou `df --output=…`, `ss -H` (sans en-tête), `free -b`, `systemctl is-active` et `systemctl is-enabled`, *etc.*
— Certaines informations sont plus simples à obtenir en lisant directement un fichier (`File.read`) qu'en

analysant la sortie d'une commande : `/proc/uptime`, `/proc/loadavg`, `/proc/meminfo`, `/etc/os-release`, `/etc/passwd`.
- Structurez le script avec une méthode par information, renvoyant un `Hash` ; le résultat global est un `Hash` que l'on affiche ou que l'on convertit en JSON. Une commande absente ou en échec (code de retour non nul) ne doit pas interrompre le script : consigner l'erreur dans le résultat.

3 Déploiement dans un environnement Docker

Objectif : mettre à disposition l'outil d'audit dans un conteneur Docker, ce conteneur devant auditer le **système hôte** (et non lui-même).

Dans ce cadre, le travail demandé consiste à :

1. créer un `Dockerfile` permettant :
   - d'installer Ruby et les paquets fournissant les commandes utilisées par le script,
   - de copier le script développé,
   - de définir un point d'entrée (`ENTRYPOINT`) exécutant le script réalisé, de sorte que les arguments passés à `docker run` lui soient transmis ;
2. exécuter le conteneur sans option particulière, comparer le résultat à celui obtenu directement sur l'hôte et expliquer les différences constatées ;
3. déterminer les options d'exécution (ligne de commande `docker run` et/ou fichier `compose.yaml`) permettant au conteneur d'auditer l'hôte, en justifiant chacune d'entre elles et en discutant de leurs implications en termes de sécurité.

Indications
Par défaut, un conteneur dispose de ses propres *namespaces* (processus, réseau, nom de machine, système de fichiers) : il ne voit que ses propres processus, interfaces et fichiers. En revanche, le noyau est partagé avec l'hôte : certaines informations (`uname -r`, `/proc/uptime`, `/proc/loadavg`, *etc.*) sont donc celles de l'hôte, même sans option particulière. Les pistes suivantes permettent d'accéder aux informations de l'hôte :

Information	Piste
processus	`--pid=host`
interfaces et connexions réseau	`--network=host`
nom de la machine	`--uts=host`
fichiers de l'hôte (`/etc`, *etc.*)	montage de `/` en lecture seule (`-v /:/host:ro`) et option « racine » du script
espace disque	`df` sur les points de montage vus sous `/host` ; la liste des montages de l'hôte est visible dans `/proc/1/mounts` (avec `--pid=host`)
utilisateurs connectés	`who` accepte un fichier `utmp` en argument ; à défaut (versions récentes), fichiers de session de `systemd-logind` dans `/run/systemd/sessions/`
services	`systemctl` dialogue avec `systemd` *via* une *socket* située dans `/run/systemd` (à monter) ; voir aussi la variable `SYSTEMD_IGNORE_CHROOT`
processus des connexions (`ss -p`)	nécessite de lire `/proc/<pid>/fd` des processus de l'hôte : capacité `SYS_PTRACE` (`--cap-add`)

L'option `--privileged` est à proscrire : n'accordez que les droits strictement nécessaires et privilégiez les montages en lecture seule.

3 Pour aller plus loin (bonus)

Les éléments suivants sont **facultatifs** et seront valorisés dans l'évaluation. Pour être pris en compte, ils doivent être documentés comme ceux de la partie page 1 et intégrés au script Ruby (affichage et JSON).

- *Matériel* : modèle du processeur et nombre de cœurs (`lscpu`, `nproc`, `/proc/cpuinfo`).
- *Réseau* : route par défaut et serveurs DNS utilisés (`ip route`, `resolvectl`, `/etc/resolv.conf`).
- *Virtualisation* : détection d'une exécution dans une machine virtuelle ou un conteneur (`systemd-detect-virt`, `/.dockerenv`).
- *Sécurité* :
  - membres du groupe `sudo` (`getent group`),
  - dernières connexions et tentatives échouées (`last`, `lastb`, `journalctl -u ssh`) ; sous Debian 13, `last` s'appuie sur `wtmpdb` et non plus sur `/var/log/wtmp`,
  - fichiers possédant le bit SUID (`find / -perm -4000`).
- *Exploitation* :
  - services en échec (`systemctl --failed`),
  - tâches planifiées : *timers* `systemd` (`systemctl list-timers`) et *crontabs*,
  - erreurs du démarrage courant (`journalctl -p err -b`),
  - mises à jour disponibles (`apt list --upgradable`),
  - utilisation des *inodes* (`df -i`).
- *Sources des informations* : pour au moins quatre informations de la partie page 1, identifier le fichier de `/proc` ou de `/sys` effectivement lu par la commande utilisée (par exemple, `free` lit `/proc/meminfo`). La commande `strace` peut vous y aider.

4 Résultats attendus

- Un dépôt Git contenant le code Ruby pour l'audit, le `Dockerfile` et, le cas échéant, le fichier `compose.yaml`.
- Un mini-rapport contenant les explications de la première partie, l'analyse des différences constatées lors de l'exécution dans le conteneur et la justification des options Docker retenues.
- Une démonstration du résultat s'il est possible de l'organiser.